

Java Generics

↓
(Type safety)

Set of Rules

10 → Integer
"Hello" → String
true → Boolean

Values → Data Types

Integer → Arithmetic opn
String → String Methods
Student → Student Methods

Types → Rules

int x = 10;
 ↳ 10-2;

String s = "Hello";

int y = 8; // Error

JAVA → Typed language

int ~~X~~ = "Hello" + 5;

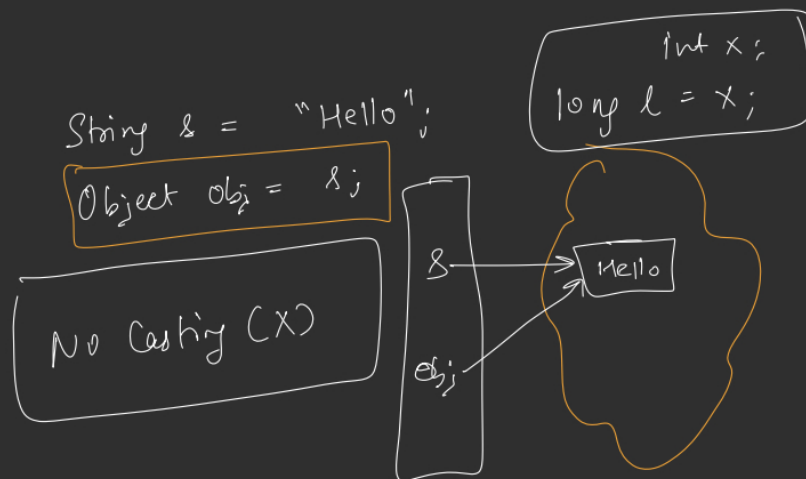
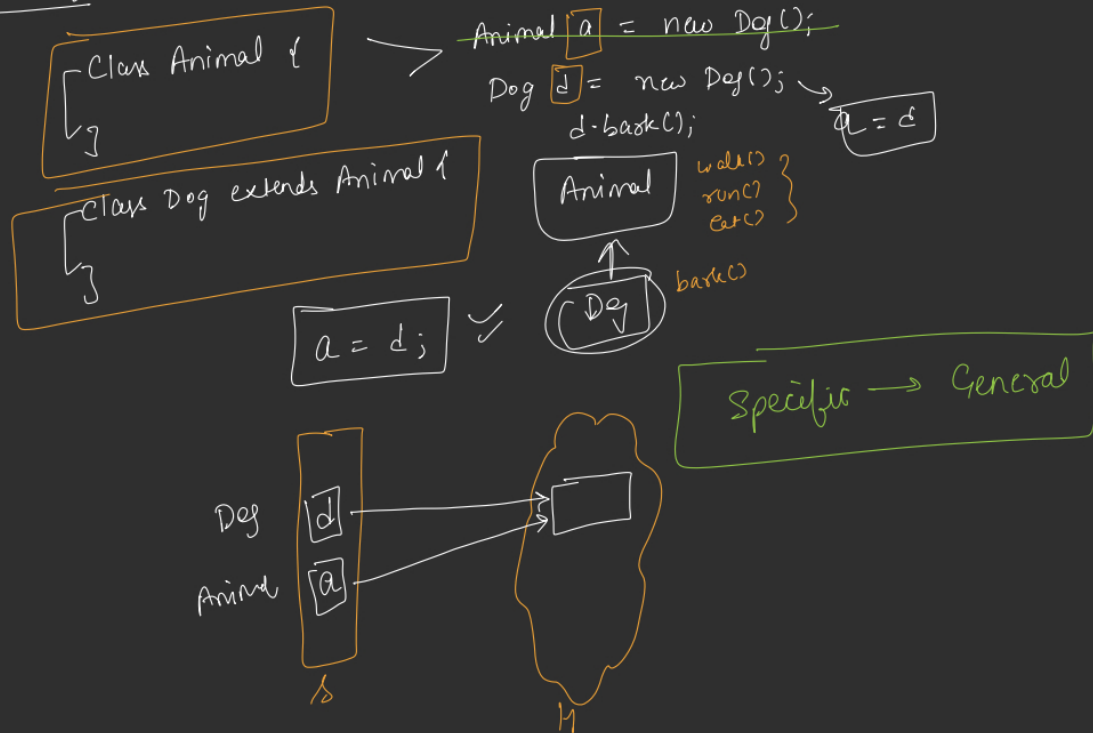
UpCasting & DownCasting

long l = 12345678;

int i = l;

int i = (int) l;

Upcasting:



Downcasting:

Object obj = "Hello";

String s = obj; // Compiler error

String s = (String) obj;

Object obj = ~~obj~~ // ind
String s = (String) obj;

App / Console

Compile Time

```
int x = "Hello";
```

RunTime

```
Object obj = 10;  
String s = (String) obj;
```

→ Production Bug

→ Code is safer

→ Better IDE Support

→ Debugging

Limitations of using Object as Universal Type

- Type information is lost.
- Wrong object could be inserted.
- Casting became necessary when reading.
- Many errors shift to Runtime.

Generics

Generics

```
Class Box<T> {  
    T value;  
}
```

Number

Integer

Double

Float

⋮

;

String

Boolean

Student

"Ability"

```
Box<String> b1 = new Box<>(" ");
```